

ATTENDY

your #1 attendance manager

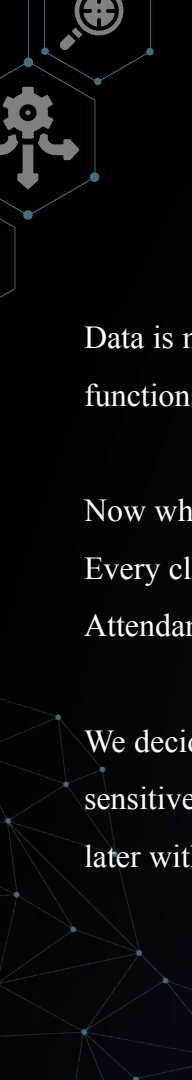
Al Wahhaj, Khalil, Al Azhar, Mahboob, Mohammed Al Balushi

Year 1

ATTENDY

Classes - Listed & Explained

<i>Attendance Manager</i>	<i>Attendance Record</i>	<i>Course</i>	<i>Report Helper</i>	<i>Instructor</i>	<i>Person</i>	<i>Student</i>	<i>Main</i>
Controls the system menu and manages students, instructors, courses, and attendance operations.	Stores a student's attendance status for a specific date.	Manages a course's students, instructor, attendance records, and reports.	Calculates and prints student attendance statistics and make sure the program won't crashes.	Stores instructor details such as department and assigned course.	Stores the shared name and ID information for students and instructors.	Stores student details such as program and study year.	Creates Attendance Manager Object and runs the program.



Encapsulation

Data is marked private. The only way to read or change it is through public functions we choose to provide (getter and setter functions for example).

Now where did we use encapsulation?

Every class in our project: Course, AttendanceManager, Student, Instructor, AttendanceRecord, all use private data with public getters and setters.

We decide what outsiders can see and change (because Attendy has so much sensitive data). Keeps the data safe and makes it easier to change internal stuff later without breaking everything else.

```
class AttendanceManager {  
private:  
    vector<Student*> students;  
    vector<Instructor*> instructors;  
    vector<Course*> courses;
```

*from attendancemanager.h
lines 13-17*

```
class Course {  
private:  
    string courseCode;  
    string courseName;  
    Instructor* instructor;  
    vector<Student*> students;  
    vector<AttendanceRecord> attendanceRecords;  
public:  
    //constructors and destructors  
    Course();  
    Course(string courseCode, string courseName);  
  
    ~Course();
```

*from Course.h
lines 10-19*



Constructors & Destructors

We're looping through all the students, instructors, and courses we stored, and deleting each one to free up memory.

Without this destructor, every new we used would leave memory behind and we will run out of RAM until the program crashes (lack of memory).

This keeps our system clean and efficient, because good memory management is required ESPECIALLY for big programs.

As for the constructor, it automatically loads pre-set default courses when the manager is created. No need to call anything manually.

```
AttendanceManager::~AttendanceManager() {  
    for (Student* student : students) { // cleaning memory  
        delete student;  
    }  
    for (Instructor* instructor : instructors) {  
        delete instructor;  
    }  
    for (Course* course : courses) {  
        delete course;  
    }  
}
```

*from attendancemanager.cpp
lines 14-23*

```
AttendanceManager::AttendanceManager() {  
    loadDefaultCourses();  
}
```

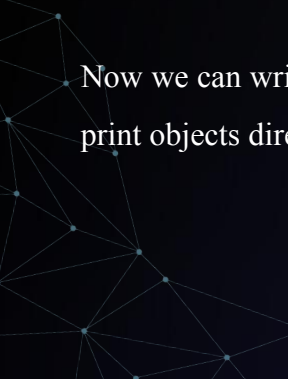
*from attendancemanager.h
line 19*



Operator Overloading

It lets us print an AttendanceRecord directly using `cout << record` instead of calling another function (like `record.print()`, or whatever it is).

We tell C++ what `<<` means when used with our class...it takes the student ID, date, and status and sends them to the terminal in a nice format (as seen in reference picture).



Now we can write `cout << record` and it looks clean because being able to print objects directly makes life easier!

```
ostream& operator<<(ostream& out, const AttendanceRecord& record) {  
    out << "Student ID: " << record.studentId  
    << " | Date: " << record.date  
    << " | Status: " << record.status;  
    return out;  
}
```

*from AttendanceRecord.cpp
lines 61-67*



Composition & Pointers

In this case, we KNOW that a course has instructors, students, and an attendance record. This is a has-a relationship (aka, composition).

The instructor and students show aggregation because they are stored as pointers, meaning they can exist outside the course. However, the attendance records show composition because they are stored directly inside the Course class and belong to it.

This keeps the code clean and short, without repetition and a mess.

```
class Course {  
private:  
    string courseCode;  
    string courseName;  
    Instructor* instructor;  
    vector<Student*> students;  
    vector<AttendanceRecord> attendanceRecords;
```

*from Course.h
lines 10-16*

Exception Handling

When we call ReportHelper functions to count total and present classes, and calculate percentages, we put each operation inside a try block. If something is wrong (for example: empty student ID or null pointer) the validation functions throw an exception and we catch it right there.

In ReportHelper.cpp, inside countTotalClasses() (ref pic) we validate the student ID first, then do the counting. If validation fails, an exception gets thrown.

Without this, an empty student ID or bad pointer would crash the whole program.

*from ReportHelper.cpp
lines 7-17*

```
void ReportHelper::validateStudentId(const string& studentId) {  
    if (studentId.empty()) {  
        throw invalid_argument("student ID can not be empty") ;  
    }  
}  
  
void ReportHelper::validateStudentPointer(const Student* student) {  
    if (student == nullptr) {  
        throw invalid_argument("the student pointer is empty");  
    }  
}
```

```
int ReportHelper::countTotalClasses(const string& studentId, //here we  
                                   const vector<AttendanceRecord>& records) {  
    try { //check  
        validateStudentId(studentId);  
  
        int total=0;  
        for (const AttendanceRecord& record : records) { //loop around  
            if (record.getStudentId() ==studentId) { //add for each  
                total++;  
            }  
        }  
        return total;  
    }  
    catch (const invalid_argument& error) {  
        throw; //we used catch 2 to  
    } //different type of  
    catch (const exception& error) {  
        throw runtime_error("Error when counting all classes: " + string(error.what()));  
    }  
}
```

*from ReportHelper.cpp
lines 18-38*



Inheritance

Instead of rewriting name and id in every type of person, we put them in Person once. Then Instructor just says : public Person and gets everything (then adds its own department and course, ref pic).

Parent class is Person, and child class is Instructor. Obviously, an instructor is a person so it makes sense to inherit a person's attributes for the instructor class.

Inheritance is about reusing code by modeling real world relationships. If we want to fix or update anything in the person class, all the classes inheriting from it will be updated too.

```
Instructor::Instructor() : Person() {  
    department = "Unknown";  
    course = "Unknown";  
}  
  
Instructor::Instructor(string name, string id, string department, string course)  
    : Person(name, id) {  
    this->department = department;  
    this->course = course;  
}
```

*from Instructor.cpp
lines 7-16*



Polymorphism

We marked `displayInfo()` as `virtual` in `Person`, then overrode it in `Student` and `Instructor` using the `override` keyword.

`Person.h` has the virtual function. `Student.cpp` and `instructor.cpp` have their own versions.

Now if we call `displayInfo()` on a `Person*` pointer, C++ automatically runs the correct version. Students show their program, instructors show their department. It is the same function name with different behavior and that is polymorphism!

```
void displayInfo() const override;
```

*from Student.h
line 27
(override declaration)*

```
public:  
    Person();  
    Person(string name, string id);  
  
    virtual ~Person();  
  
    string getName() const;  
    void setName(string name);  
  
    string getId() const;  
    void setId(string id);  
  
    virtual void displayInfo() const;  
};
```

*from Person.h
lines 17 and 25
(virtual function declaration)*

```
void Student::displayInfo() const {  
    cout << "Student Name: " << name << endl;  
    cout << "ID: " << id << endl;  
    cout << "Program: " << program << endl;  
    cout << "Year: " << year << endl;  
}
```

*from Student.cpp
lines 44-49
(override implementation)*



Thanks for watching! Any questions?

Presentation was done to you by Al Wahhaj, Mahboob, Al Azhar, Khalil, & Mohammed Al Balushi

